

Rapport final

Objet connecté pour la détection d'objets (fruits) avec Deep Learning et MQTT

1. Introduction

Ce projet s'inscrit dans le cadre de la réalisation d'un objet connecté intelligent capable de :

- ✓ détecter des fruits en temps réel à l'aide du Deep Learning et d'une webcam,
- ✓ transmettre les résultats de détection vers un broker MQTT cloud,
- ✓ stocker et analyser les données dans une base de données,
- ✓ permettre à une application web d'envoyer des commandes (ouvrir/fermer une porte, activer/désactiver une alarme) vers un objet/actionneur via MQTT.

L'architecture globale repose sur trois grandes étapes :

1. Détection d'objets par réseau de neurones (YOLO)
2. Communication MQTT (publication / souscription)
3. Application web avec base de données et commandes IoT

2. Étape 1 : Détection d'objets par Deep Learning

2.1 Étude théorique

La détection d'objets est réalisée à l'aide de **réseaux de neurones convolutifs (CNN)**. Ces réseaux sont capables d'extraire automatiquement des caractéristiques visuelles à partir des images (formes, textures, couleurs) afin d'identifier et localiser des objets.

Dans ce projet, j'utilise le modèle YOLOv8 (You Only Look Once). Par défaut, YOLOv8 est capable de détecter un grand nombre d'objets différents (personnes, voitures, animaux, objets du quotidien, etc.), car il est pré-entraîné sur des datasets riches comme COCO.

Cependant, l'objectif de mon projet n'est pas de détecter tous les objets, mais uniquement les fruits. J'ai donc adapté le modèle afin de filtrer et conserver uniquement les classes correspondant aux fruits, et d'ignorer toutes les autres détections.

2.2 Spécialisation du modèle pour la détection des fruits

Dans ce projet, j'ai utilisé YOLOv8, un modèle capable de détecter de nombreux objets différents (personnes, cat, etc.). Toutefois, mon objectif n'était pas de détecter tous ces objets, mais uniquement les fruits.

J'ai donc décidé de filtrer les classes détectées afin d'ignorer tous les objets non pertinents et de conserver uniquement les fruits. Ensuite, j'ai choisi d'ajouter d'autres types de fruits pour enrichir le projet et le rendre plus réaliste.

Concrètement, j'ai :

- ✓ éliminé les objets non liés aux fruits (personnes, cat,...),
- ✓ conservé et ajouté plusieurs classes de fruits (banana, apple, orange, grape, mango, etc.),
- ✓ adapté le code pour traiter uniquement ces classes.

Ce choix m'a permis :

- ✓ d'améliorer la pertinence des résultats,
- ✓ de réduire les fausses détections,
- ✓ d'adapter le système à un cas d'usage précis : la détection intelligente de fruits.

Dans le code, j'ai défini une liste de classes autorisées :

```
FRUIT_CLASSES = [46, 47, 49] # banana, apple, orange
```

Ainsi, je n'envoie via MQTT que les détections correspondant aux fruits, avec un seuil de confiance suffisant.

2.3 Dataset et classes

Un fichier data.yaml définit :

- ✓ les chemins des données d'entraînement et de validation,
- ✓ le nombre de classes,
- ✓ les noms des fruits à détecter.

Exemple de classes définies :

- ✓ avocado
- ✓ cherry
- ✓ grape
- ✓ apple
- ✓ orange

2.4 Détection en temps réel et publication MQTT

Le script detect_and_publish.py permet de :

- ✓ ouvrir la webcam,
- ✓ détecter uniquement les fruits sélectionnés,
- ✓ afficher les bounding boxes et le taux de confiance,
- ✓ publier les résultats de détection vers un broker MQTT.

3. Étape 2 : Communication MQTT (Broker Cloud)

3.1 Choix du broker

Le broker utilisé est HiveMQ Cloud, un broker MQTT public et sécurisé, accessible via TLS.

Les topics utilisés sont :

- ✓ `iot/detection` : publication des résultats de détection
- ✓ `iot/command` : réception des commandes envoyées depuis l'application web

3.2 Publication des données

Le PC exécutant YOLO agit comme publisher MQTT. À chaque détection valide, un message JSON est publié contenant :

- ✓ l'objet détecté,
- ✓ le taux de confiance,
- ✓ la date et l'heure.

3.3 Souscription et stockage des données

Un script mqtt_subscriber_sqlite.py agit comme subscriber et :

- ✓ reçoit les messages du topic iot/detection,
- ✓ extrait les informations,
- ✓ les stocke dans une base de données SQLite.

Structure de la base de données :

```
CREATE TABLE detections (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  topic TEXT,  
  objet TEXT,  
  confiance REAL,  
  received_at TEXT  
);
```

→ Messages reçus via MQTT et insertion dans la base SQLite.

3.4 Simulation de l'actionneur

Un script mqtt_command_subscriber.py simule un objet IoT/actionneur. Il est abonné au topic iot/command et réagit aux commandes :

- ✓ ouverture / fermeture de porte,
- ✓ activation / désactivation d'une alarme.

4. Étape 3 : Application Web IoT

4.1 Technologie utilisée

L'application web a été développée avec Streamlit (Python). Elle permet une visualisation simple, rapide et interactive des données.

4.2 Fonctionnalités principales

L'application web permet de :

- ✓ afficher les statistiques des objets détectés,
- ✓ visualiser les dernières détections (objet, confiance, date),
- ✓ envoyer des commandes IoT via MQTT.

4.3 Interface utilisateur

L'interface contient :

- ✓ un titre principal,
- ✓ une section Statistiques (nombre de détections par fruit),
- ✓ un tableau des dernières détections,
- ✓ une section Commandes IoT avec des boutons.

→ Interface Streamlit avec statistiques et tableau.

4.4 Commandes IoT

Chaque bouton envoie une commande MQTT vers le topic `iot/command` :

- ✓ 🚪 Ouvrir la porte → { "device": "door", "action": "OPEN" }
- ✓ 🔒 Fermer la porte → { "device": "door", "action": "CLOSE" }
- ✓ 📞 Activer l'alarme → { "device": "alarm", "action": "ON" }
- ✓ 🔔 Désactiver l'alarme → { "device": "alarm", "action": "OFF" }

5. Architecture globale du système

- ✓ Webcam + YOLO → Détection des fruits
- ✓ Publisher MQTT → Envoi des résultats
- ✓ Broker MQTT Cloud (HiveMQ)
- ✓ Subscriber + SQLite → Stockage des données
- ✓ Application Web (Streamlit) → Visualisation & commandes
- ✓ Actionneur MQTT → Exécution des commandes

6. Les fichiers et les captures :

Le fichier `iot_backend.py`

```
from ultralytics import YOLO

import cv2

import paho.mqtt.client as mqtt

import json

import sqlite3

from datetime import datetime

# CONFIGURATION MQTT

BROKER = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"

PORT = 8883

USERNAME = "wiemchedly"

PASSWORD = "Wiemlovemama123#"
```

```
TOPIC_DETECTION = "iot/detection"

TOPIC_COMMAND = "iot/command"

# BASE DE DONNÉES SQLITE

conn = sqlite3.connect("iot.db", check_same_thread=False)

cursor = conn.cursor()

# Création de la table pour stocker les détections

cursor.execute("""

CREATE TABLE IF NOT EXISTS detections (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    objet TEXT,

    confiance REAL,

    received_at TEXT

)

""")

conn.commit()

print("✓ Backend IoT prêt")

# CALLBACKS MQTT

def on_connect(client, userdata, flags, rc):

    if rc == 0:
```

```
print("✓ Connecté au broker MQTT")

client.subscribe(TOPIC_COMMAND) # S'abonner au topic des commandes

else:

    print("✗ Erreur de connexion MQTT")

def on_message(client, userdata, msg):

    data = json.loads(msg.payload.decode())

    if msg.topic == TOPIC_COMMAND:

        device = data.get("device")

        action = data.get("action")

        if device == "door":

            print("🚪 Porte :", action)

        elif device == "alarm":

            print("🚨 Alarme :", action)

# CLIENT MQTT

mqtt_client = mqtt.Client("IoT_Backend")

mqtt_client.username_pw_set(USERNAME, PASSWORD)

mqtt_client.tls_set()

mqtt_client.tls_insecure_set(True)

mqtt_client.on_connect = on_connect
```



```
mqtt_client.on_message = on_message

mqtt_client.connect(BROKER, PORT)

mqtt_client.loop_start() # Démarrer le thread MQTT

# CHARGEMENT DU MODÈLE YOLO

model = YOLO("C:/Users/ASUS/Desktop/Projet_IOT_Wiem/yolov8s.pt")

# Définition des classes fruits à détecter (COCO IDs)

FRUIT_CLASSES = [46, 47, 49] # banana, apple, orange

# CAPTURE DE LA WEBCAM

cap = cv2.VideoCapture(0)

print("📷 Webcam active (appuyez sur Q pour quitter)")

while True:

    ret, frame = cap.read()

    if not ret:

        break

    results = model(frame, stream=True, verbose=False)

    for r in results:

        for box in r.bboxes:

            cls_id = int(box.cls[0])

            conf = float(box.conf[0])

            # Filtrer uniquement les fruits avec confiance > 0.6
```

```
if cls_id in FRUIT_CLASSES and conf > 0.6:

    name = model.names[cls_id]

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # AFFICHAGE DE LA BOÎTE DE DÉTECTION

    x1, y1, x2, y2 = map(int, box.xyxy[0])

    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)

    cv2.putText(frame, f"{name} {conf:.2f}", (x1, y1-10),

                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

    # ENVOI DES DONNÉES SUR MQTT

    message = {

        "objet": name,

        "confiance": round(conf, 3),

        "heure": timestamp

    }

    mqtt_client.publish(TOPIC_DETECTION, json.dumps(message))

    # ENREGISTREMENT DANS SQLITE

    cursor.execute(

        "INSERT INTO detections (objet, confiance, received_at) VALUES (?, ?, ?)",

        (name, round(conf, 3), timestamp)

    )
```

```
conn.commit()

print(" 📡 Détection envoyée & enregistrée :", message)

cv2.imshow("Detection IoT", frame)

# Quitter si 'q' est pressé

if cv2.waitKey(1) & 0xFF == ord("q"):

    break

# FERMETURE PROPRE

cap.release()

cv2.destroyAllWindows()

mqtt_client.loop_stop()

mqtt_client.disconnect()

conn.close()

print(" 🛑 Backend IoT arrêté")
```

Explication du fichier :

Ce script constitue le backend IoT de notre projet. Il réalise plusieurs fonctions importantes :

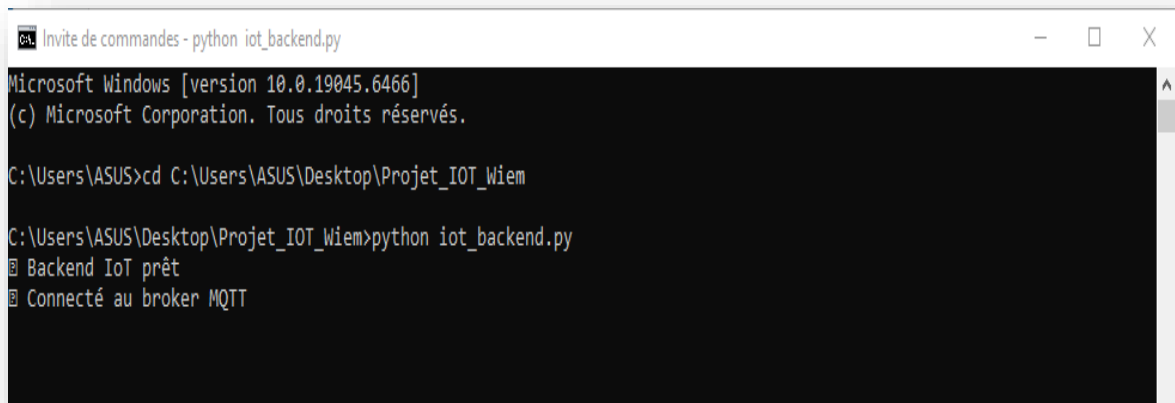
1. **Détection d'objets avec YOLOv8** : la webcam capture les images en temps réel et le modèle identifie uniquement les fruits en filtrant les autres objets.
2. **Communication MQTT** : les détections valides sont publiées dans le topic `iot/detection` et les commandes IoT (porte, alarme) sont reçues via `iot/command`.

3. **Stockage des données** : chaque détection est enregistrée dans une base SQLite locale pour conserver l'historique.
4. **Affichage visuel** : les bounding boxes et les noms des fruits détectés sont affichés à l'écran pour le suivi en temps réel.

Ce fichier centralise donc les fonctions détection, communication et stockage, et constitue le lien entre le modèle YOLO, le broker MQTT et l'application frontend.

Taper dans un terminal 1 :

Python `iot_python.py`

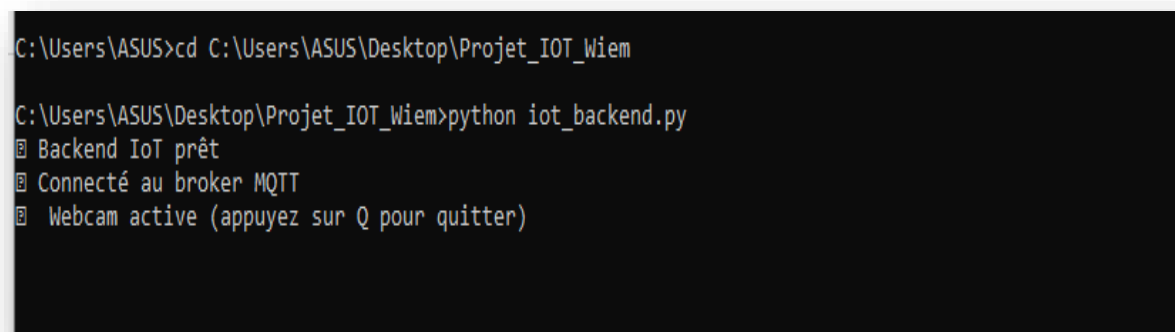


```
Invite de commandes - python iot_backend.py
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\ASUS>cd C:\Users\ASUS\Desktop\Projet_IOT_Wiem

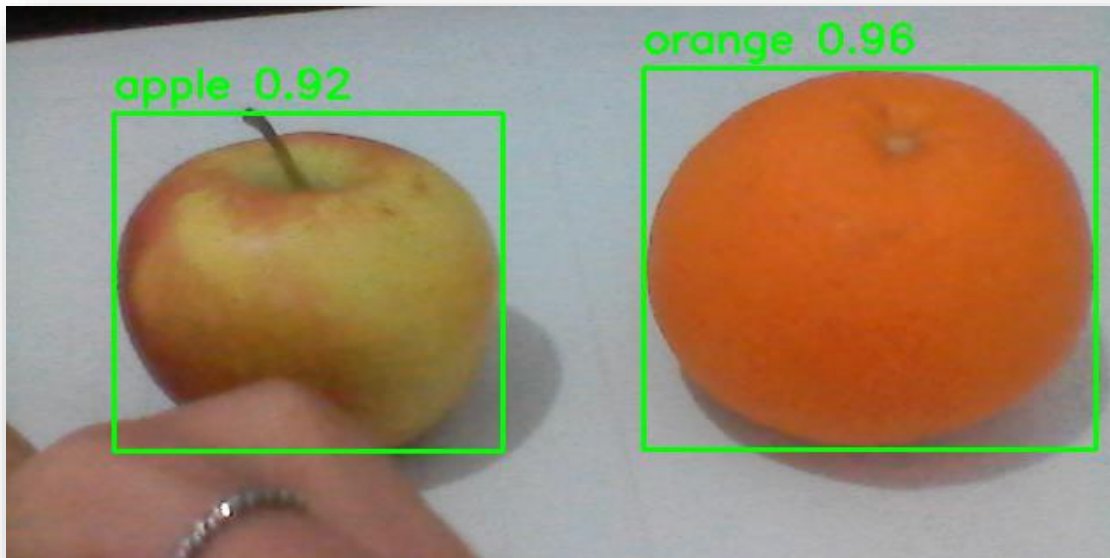
C:\Users\ASUS\Desktop\Projet_IOT_Wiem>python iot_backend.py
Backend IoT prêt
Connecté au broker MQTT
```

Résultat :



```
C:\Users\ASUS>cd C:\Users\ASUS\Desktop\Projet_IOT_Wiem

C:\Users\ASUS\Desktop\Projet_IOT_Wiem>python iot_backend.py
Backend IoT prêt
Connecté au broker MQTT
Webcam active (appuyez sur Q pour quitter)
```



```
C:\Users\ASUS\Desktop\Projet_IOT_Wiem>python iot_backend.py
Backend IoT prêt
Connecté au broker MQTT
Webcam active (appuyez sur Q pour quitter)
Détection envoyée & enregistrée : {'objet': 'orange', 'confiance': 0.935, 'heure': '2025-12-15 17:48:09'}
Détection envoyée & enregistrée : {'objet': 'apple', 'confiance': 0.862, 'heure': '2025-12-15 17:48:09'}
Détection envoyée & enregistrée : {'objet': 'orange', 'confiance': 0.922, 'heure': '2025-12-15 17:48:15'}
Détection envoyée & enregistrée : {'objet': 'apple', 'confiance': 0.858, 'heure': '2025-12-15 17:48:15'}
Détection envoyée & enregistrée : {'objet': 'orange', 'confiance': 0.934, 'heure': '2025-12-15 17:48:19'}
Détection envoyée & enregistrée : {'objet': 'apple', 'confiance': 0.866, 'heure': '2025-12-15 17:48:19'}
```

Paramètres de connexion

Connectez-vous à votre cluster Cloud HiveMQ avec vos identifiants. Ou bien, connectez-vous instantanément avec des identifiants générés automatiquement.

Nom d'utilisateur: Mot de passe:

Déconnexion

Abonnements au sujet 1 Désabonnement de tous les sujets

Messages 6 **Effacer tous les messages**

Id	Sujet	QoS	Message
0	Iot/détection	0	{"objet": "orange", "confiance": 0.935, "heure": "2025-12-15 17:48:09"}
1	Iot/détection	0	{"objet": "pomme", "confiance": 0.862, "heure": "2025-12-15 17:48:09"}
2	Iot/détection	0	{"objet": "orange", "confiance": 0.922, "heure": "2025-12-15 17:48:15"}
3	Iot/détection	0	{"objet": "pomme", "confiance": 0.858, "heure": "2025-12-15 17:48:15"}

Le fichier frontend : app.py

```
import streamlit as st

import sqlite3

import paho.mqtt.client as mqtt

import json

import pandas as pd

# CONFIG PAGE

st.set_page_config(

    page_title="Plateforme IoT – Détection Fruits",

    page_icon="🍏",

    layout="wide"

)

# STYLE CSS

st.markdown("""

<style>

/* TITRE PRINCIPAL */

.main-title {

    font-size: 36px;

    font-weight: 700;

    margin-bottom: 5px;
```

```
}

/* SOUS TITRES */

.section-title {

    font-size: 23px;

    font-weight: 600;

    margin-top: 10px;

    margin-bottom: 10px;

}

/* CARTES STATISTIQUES */

.stat-card {

    background-color: #f7f7f7;

    padding: 12px;

    border-radius: 10px;

    text-align: center;

    box-shadow: 0 2px 6px rgba(0,0,0,0.05);

}

.stat-card h3 {

    font-size: 16px;

    margin-bottom: 5px;

}
```

```
.stat-card h1 {  
  
    font-size: 22px;  
  
    margin: 0;  
  
}  
  
/* TABLEAU */  
  
[data-testid="stDataFrame"] {  
  
    max-width: 75%;  
  
    margin: auto;  
  
}  
  
/* BOUTONS */  
  
.stButton > button {  
  
    padding: 8px 12px;  
  
    font-size: 14px;  
  
}  
  
</style>  
  
""", unsafe_allow_html=True)  
  
# TITRE  
  
st.markdown("<div class='main-title'>🍷🍏 Plateforme IoT – Détection de Fruits</div>",  
unsafe_allow_html=True)
```



```
st.caption("Surveillance en temps réel & commandes IoT")

st.divider()

# MQTT (COMMANDES)

broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"

port = 8883

username = "wiemchedly"

password = "Wiemlovemama123#"

topic_cmd = "iot/command"

mqtt_client = mqtt.Client("Streamlit_CMD")

mqtt_client.username_pw_set(username, password)

mqtt_client.tls_set()

mqtt_client.tls_insecure_set(True)

mqtt_client.connect(broker, port)

# DATABASE

conn = sqlite3.connect("iot.db", check_same_thread=False)

cursor = conn.cursor()

cursor.execute("""

SELECT objet, confiance, received_at

FROM detections

ORDER BY received_at DESC
```

```
""")

rows = cursor.fetchall()

df = pd.DataFrame(rows, columns=["Objet", "Confiance", "Date & Heure"])

# STATISTIQUES

st.markdown("<div class='section-title'>📊 Statistiques</div>", unsafe_allow_html=True)

if not df.empty:

    stats = df["Objet"].value_counts()

    cols = st.columns(len(stats))

    for col, (obj, count) in zip(cols, stats.items()):

        with col:

            st.markdown(f""""

            <div class="stat-card">

                <h3>{obj}</h3>

                <h1>{count}</h1>

                <p>détections</p>

            </div>

            """, unsafe_allow_html=True)

else:

    st.info("Aucune donnée disponible")

st.divider()
```

```
# DERNIÈRES DÉTECTIONS

st.markdown("<div class='section-title'>📁 Dernières détections</div>",
unsafe_allow_html=True)

if not df.empty:

    st.dataframe(df.head(10), use_container_width=False)

else:

    st.warning("Pas encore de détection")

st.divider()

st.markdown("<div class='section-title'>📁 Commandes IoT</div>",
unsafe_allow_html=True)

col1, col2 = st.columns(2)

with col1:

    if st.button("🚪 Ouvrir la porte"):

        mqtt_client.publish(

            topic_cmd,

            json.dumps({"device": "door", "action": "OPEN"})

        )

        st.success("Commande envoyée : OUVRIR la porte")

    if st.button("🔒 Fermer la porte"):
```

```
mqtt_client.publish(

    topic_cmd,

    json.dumps({"device": "door", "action": "CLOSE"})

)

st.success("Commande envoyée : FERMER la porte")

with col2:

    if st.button("🔔 Activer l'alarme"):

        mqtt_client.publish(

            topic_cmd,

            json.dumps({"device": "alarm", "action": "ON"})

        )

        st.success("Commande envoyée : ALARME ACTIVÉE")

    if st.button("🔕 Désactiver l'alarme"):

        mqtt_client.publish(

            topic_cmd,

            json.dumps({"device": "alarm", "action": "OFF"})

        )

        st.success("Commande envoyée : ALARME DÉSACTIVÉE")

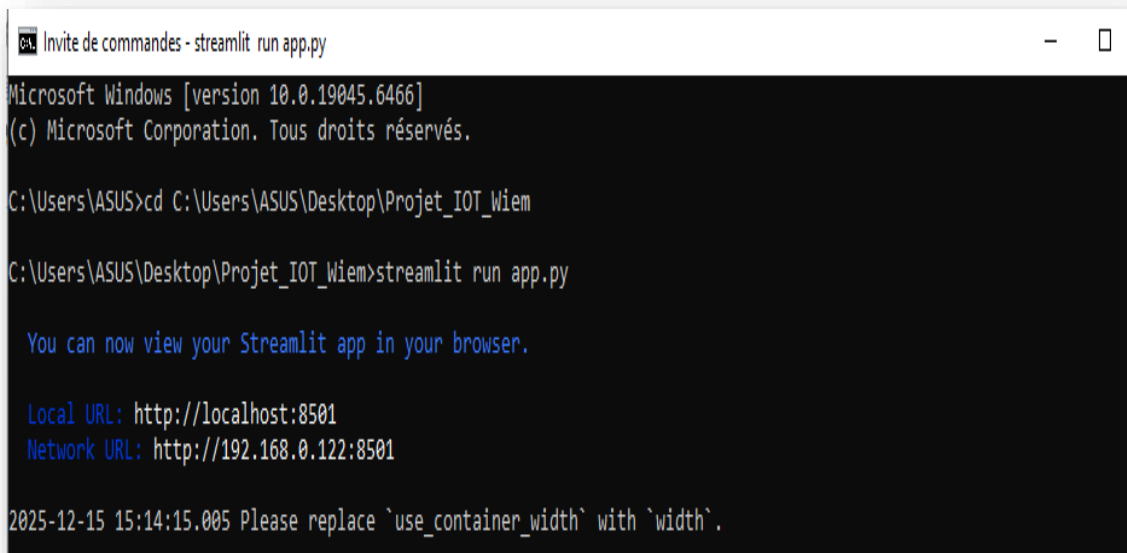
st.divider()

st.caption("🐼 Projet IoT – YOLO • MQTT • SQLite • Streamlit")
```

Le fichier `app.py` constitue le frontend de notre projet IoT. Il est développé avec Streamlit et sert à visualiser en temps réel les détections de fruits effectuées par le modèle YOLOv8 et enregistrées dans la base de données SQLite. L'application affiche à la fois les statistiques des détections (nombre de fois qu'un fruit a été détecté) et les dernières détections sous forme de tableau. De plus, elle permet à l'utilisateur d'envoyer des commandes IoT (ouvrir ou fermer une porte, activer ou désactiver une alarme) via le broker MQTT. Chaque bouton publie une commande sur le topic approprié et affiche une confirmation visuelle dans l'interface. Ainsi, `app.py` combine visualisation, interaction et contrôle des objets IoT, offrant une interface simple et intuitive pour suivre et commander le système connecté.

Taper dans un terminal 2 :

Streamlit run `app.py`



```
Invite de commandes - streamlit run app.py
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\ASUS>cd C:\Users\ASUS\Desktop\Projet_IOT_Wiem

C:\Users\ASUS\Desktop\Projet_IOT_Wiem>streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.122:8501

2025-12-15 15:14:15.005 Please replace `use_container_width` with `width`.
```

Résultat :

 **Dernières détections**

	Objet	Confiance	Date & Heure
0	orange	0.934	2025-12-15 17:48:19
1	apple	0.866	2025-12-15 17:48:19
2	orange	0.922	2025-12-15 17:48:15
3	apple	0.858	2025-12-15 17:48:15
4	orange	0.935	2025-12-15 17:48:09
5	apple	0.862	2025-12-15 17:48:09

 Commandes IoT Ouvrir la porte Fermer la porte Activer l'alarme Désactiver l'alarme

```

C:\Users\ASUS>cd C:\Users\ASUS\Desktop\Projet_IOT_Wiem
C:\Users\ASUS\Desktop\Projet_IOT_Wiem>python iot_backend.py
Backend IoT prêt
Connecté au broker MQTT
Webcam active (appuyez sur Q pour quitter)
Détection envoyée & enregistrée : {'objet': 'orange', 'confiance': 0.935, 'heure': '2025-12-15 17:48:09'}
Détection envoyée & enregistrée : {'objet': 'apple', 'confiance': 0.862, 'heure': '2025-12-15 17:48:09'}
Détection envoyée & enregistrée : {'objet': 'orange', 'confiance': 0.922, 'heure': '2025-12-15 17:48:15'}
Détection envoyée & enregistrée : {'objet': 'apple', 'confiance': 0.858, 'heure': '2025-12-15 17:48:15'}
Détection envoyée & enregistrée : {'objet': 'orange', 'confiance': 0.934, 'heure': '2025-12-15 17:48:19'}
Détection envoyée & enregistrée : {'objet': 'apple', 'confiance': 0.866, 'heure': '2025-12-15 17:48:19'}
Alarme : ON
Alarme : OFF
Porte : OPEN
Porte : CLOSE

```

```

0  Sujet : IOT/commandement  QoS : 0
   {"dispositif" : « alarme », « action » : « ACTIVÉ"}
1  Sujet : IOT/commandement  QoS : 0
   {"dispositif" : « alarme », « action » : « ÉTEINT"}
2  Sujet : IOT/commandement  QoS : 0
   {"dispositif" : « porte », « action » : « OUVRIR"}
3  Sujet : IOT/commandement  QoS : 0
   {"dispositif" : « porte », « action » : « FERMER"}

```

7. Conclusion

Ce projet a permis de mettre en œuvre une solution complète d'objet connecté intelligent, combinant :

- ✓ Deep Learning pour la vision par ordinateur,
- ✓ communication MQTT pour l'IoT,
- ✓ base de données pour l'historique et l'analyse,
- ✓ application web interactive.

Les objectifs du sujet ont été atteints avec succès. Des améliorations futures peuvent inclure :

- ✓ automatisation des commandes selon les objets détectés,
- ✓ ajout de notifications (email, mobile),
- ✓ déploiement sur un système embarqué (Raspberry Pi).

Mots-clés : IoT, Deep Learning, YOLO, MQTT, Streamlit, SQLite